

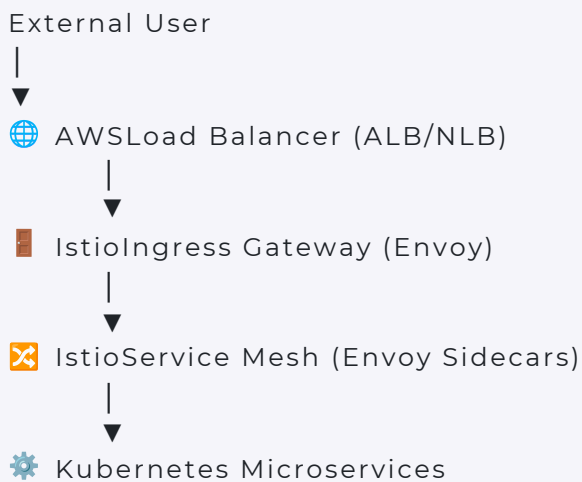


Istio on AWS EKS – Traffic Flow Explained

- A step-by-step overview of how external and internal traffic flows when using Istio Service Mesh in an Amazon EKS cluster.



Architecture Overview



- Istio controls **traffic, security, and observability** between services using **Envoy sidecar proxies**.



Istio Setup in Your EKS Cluster

- First, install **Istio** in your **Amazon EKS cluster**.
- Istio automatically injects **Envoy sidecar proxies** into each microservice pod.
- These proxies allow Istio to control:
 - 🚦 Traffic routing
 - 🛡️ Security (mTLS)
 - 📊 Observability
 - ⚖️ Load balancing
- Each microservice gets a **sidecar container** that manages network communication.
- Example Pod:

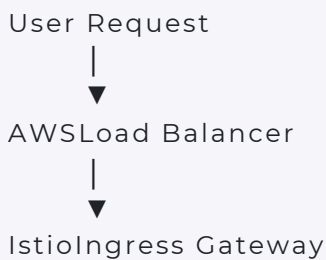
Pod

- └─ Application Container
- └─ Envoy Sidecar Proxy

■ AWS Load Balancer (External Traffic Entry)

- To expose services to the internet, an **AWS Load Balancer** is used.
- Common options:
 - 🌐 **Application Load Balancer (ALB)**
 - ⚡ **Network Load Balancer (NLB)**
- When the **Istio Ingress Gateway** is deployed, it is exposed using an AWS Load Balancer.

External Traffic Flow



Key Points

- ✅ Users send requests to the **AWS Load Balancer**
- ✅ The load balancer forwards traffic to the **Istio Ingress Gateway**
- ✅ The gateway processes HTTP/HTTPS requests

■ Istio Ingress Gateway Configuration

- The **Istio Ingress Gateway** acts as the **entry point to the service mesh**.
- You configure routing using:
 - **Gateway**
 - **VirtualService**

Gateway

- Defines how traffic enters the mesh.
- Example:

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
spec:
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
```

Virtual Service

- Controls **where the traffic goes inside the cluster**.
- Example routing:

```
route:
- destination:
    host: frontend
```

- The **Ingress Gateway** reads these rules and forwards traffic to the correct service.

NO GRAPH

Internal Traffic in Istio (Service Mesh)

- After traffic enters the cluster through the **Ingress Gateway**, it moves through the **Istio service mesh**.
- Istio manages communication between services using **Envoy sidecar proxies**.
- Capabilities:
 -  Load balancing
 -  Retries
 -  Circuit breaking
 -  mTLS encryption
 -  Monitoring

NO GRAPH

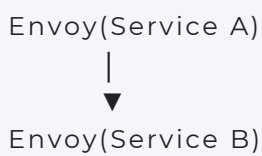
Internal Traffic Flow (Step-by-Step)

- Example: **Service A → Service B**
- Step 1

- Service A sends a request to Service B.
- Step 2
 - The **Envoy sidecar proxy in Service A** intercepts the request.



- Step 3
 - The request is routed through Istio policies and sent to **Service B's Envoy proxy**.



- Step 4
 - Service B's proxy forwards the request to the **correct pod**.

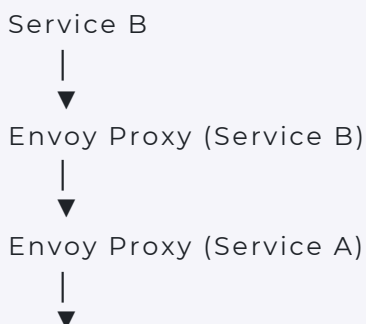
Security

- Istio can enforce **Mutual TLS (mTLS)**:
 - 🔒 Encrypts service communication
 - 🪪 Verifies service identity
 - 🛡️ Enables zero-trust networking



Response Back to External Client

- After processing the request, the response travels back through the mesh.
- Response Flow



IstioIngress Gateway



AWSLoad Balancer



External Client

Step-by-Step Response

-  Service B sends response to Service A
-  Envoy proxy handles response routing
-  Response reaches **Istio Ingress Gateway**
-  AWS Load Balancer sends response to the **external user**



End-to-End Traffic Flow Summary

Client



AWS Load Balancer



IstioIngress Gateway



ServiceMesh (Envoy Sidecars)



Microservices



Response → Gateway → Load Balancer → Client



Key Benefits of This Architecture



Security

- Mutual TLS between services



Traffic Control

- Canary deployments
- Traffic splitting
- Fault injection



Observability

- Prometheus metrics
- Jaeger tracing
- Kiali visualization



Reliability

- Retries
- Circuit breaking
- Timeouts



Final Takeaway

- When running **Istio on AWS EKS**, the architecture works like this:
- **External traffic**

User → AWS Load Balancer → Istio Ingress Gateway

- **Internal traffic**

Service → Envoy Proxy → Envoy Proxy → Service

- Istio provides a **powerful service mesh layer** that handles:
 - traffic routing
 - service security
 - observability
 - reliability
- All **without changing application code**.



DevOps Tip

- Use **Kiali** to visualize the service mesh:

```
istioctl dashboard kiali
```

- You will see:
 - service dependencies
 - traffic flow
 - error rates
 - latency
 - security status